

AMAN Codex Chat Handoff

This document is a shareable handoff compiled from the Codex session history available in this workspace. It is intended to give another user or another Codex session enough context to continue the project with similar understanding.

Important note

- This is not an official platform-level export of the full Codex conversation UI.
- It is a structured handoff built from the session context and project work.
- For continuity, this document should be shared together with the codebase.

Project identity

- Project name: AMAN
- Main mobile app path: 'AMAN/mobile_app'
- Admin web path: 'AMAN/admin_web'
- Backend: Supabase
- Mobile stack: Flutter
- Admin stack: React + Vite + Supabase

Core architecture

Authentication

- Supabase Auth stores real credentials using email and password.
- The app also uses a 'public.user' table to store business profile data:
 - 'user_id'
 - 'email'
 - 'username'
 - 'role'
 - 'full_name'
 - 'phone'
 - 'id_number'
- Login uses username in the UI, then looks up the email from 'public.user', then signs in to Supabase Auth using email and password.
- Registration creates the Supabase auth user first, then upserts profile data into 'public.user'.
- Role-based navigation sends 'owner' to owner home and all others to seeker home.

Property management

- Owner add/update/delete is handled via page -> controller -> repository.
- Property rows are stored in 'properties'.
- Property images are stored in 'property_images' and the files themselves are stored in the Supabase storage bucket 'property-images'.
- Certificates are stored in the Supabase storage bucket 'property-certificates'.
- Add property flow:
 1. Validate form
 2. Upload certificate
 3. Insert property row
 4. Upload property images
 5. Insert property image rows

- Update property flow:
 1. Load property for current owner
 2. Update price/location/description/status
 3. Upload any newly attached images
 4. Insert those new image rows
- Delete property flow:
 1. Confirm in UI
 2. Remove property image files from storage
 3. Remove leftover files under the property folder path
 4. Remove certificate file
 5. Delete property row

Seeker behavior

- Seeker home shows active properties from the database.
- Search uses RPC-backed property filtering.
- Property details page supports images and location navigation.
- Guests can browse/search/view details but other actions require login.

Recommendation / wished property

- The recommendation feature is used when the seeker does not find the property they want.
- The seeker saves desired property criteria into 'wished_property'.
- A notification/matching flow was built to match wishes against properties.
- Exact SQL matching existed before and was later disabled while testing AI matching.

Messaging and deals

- Contact from property cards opens the relevant owner/seeker chat.
- Chat behavior, unread messages, and deal completion were implemented over multiple iterations.
- Deals can drive property status changes to inactive after completion.
- Ratings were discussed and wired after deal completion, with several database adjustments along the way.

Admin dashboard

- A separate admin web project was created under 'AMAN/admin_web'.
- Stack: React + Vite + Supabase.
- Supabase connection is provided through a '.env' file using:
 - 'VITE_SUPABASE_URL'
 - 'VITE_SUPABASE_ANON_KEY'
- The admin dashboard includes a login flow and real counts pulled from Supabase.
- Hosting recommendation was to use Vercel, Netlify, or Cloudflare Pages rather than trying to host the React site directly on Supabase.

Major chronological history

Below is a chronological summary of the main requests, fixes, and architectural decisions made during this session history.

1. Profile and edit pages

- Edit information page and profile flow were built/refined.

- Seeker and owner navigation around profile and edit screens were adjusted.
- State management was repeatedly requested and preserved during changes.

2. Owner follow-up property page

- The follow-up property page was made owner-specific.
- It now shows only properties linked to the logged-in owner.
- The card header shows property type and state/city.
- Bedroom and bathroom counts were shown numerically.
- Update was linked to the specific property.
- Delete was wired.
- Image attachment for update/add was expanded to support multiple images.

3. Property deletion and storage cleanup

- Delete logic was extended so property images in storage are deleted when a property is removed.
- Additional attempts were made to ensure the virtual folder path 'properties/{ownerId}/{propertyId}' is cleaned by removing remaining files under that path.
- Certificate cleanup on property delete was also implemented.

4. State management cleanup

- The codebase was reviewed to ensure state management was consistently applied in auth, profile, property, notifications, deals, and wished-property flows.

5. Seeker home page wiring

- Properties from the database were wired into seeker home page cards.
- Cards were made responsive and show:
 - first image
 - bedrooms
 - bathrooms
 - owner name
 - rating
- Search/filter dropdowns for city and type were wired so changing the value filters the displayed results.

6. Property details page improvements

- Each property card was linked to its property details page.
- A new area row was added below the price row.
- Location info was made clickable to open the property location in Google Maps.
- All property images were made available to the seeker for browsing/swapping.

7. Search and search results

- Search property page was wired to search the database.
- Search results page was made to show matched results from the backend.
- The profile icon on search results was made functional.
- Search page/navigation behavior in guest mode and back-stack behavior on physical back were refined.

8. Project/database design discussions

- Wished property table design was discussed and then wired to the app.
- Recommendation page purpose was clarified: a seeker saves the exact property details they want when search has no suitable result.
- Rating table design was discussed multiple times and simplified so users rate each other, not the property.
- Chat/message table design was discussed, including whether to store message files or rows.
- SQL for removing unnecessary columns like URL/message type was discussed.

9. Saved-success dialogs

- A saved successfully confirmation UI was added after owner property add.
- Similar success behavior was later requested for seeker recommendation saving as well.

10. Guest mode

- Guest mode was revised so entering as guest goes directly to home.
- Guest users can search and view property details.
- Restricted features show a login-required message instead of forcing a redirect.
- Ability to navigate back to landing page from guest mode was added.

11. Messaging and chat behavior

- The message page was linked properly for each user.
- Contact from a property opens the chat with the corresponding owner.
- Back navigation from messages was fixed.
- Reloading/flicker behavior in messages after new sends was improved.
- Timing/unread display behavior in chats and message lists was improved.
- Last message time and unread counts were shown.
- The header was updated to show user names.
- Time formatting was refined toward chat-style times rather than raw date stamps.

12. Project rename to AMAN

- The project name was changed to AMAN.
- Necessary file and identifier changes were made around the rename.
- Guidance was given for GitHub remote setup and future pushes.

13. Recommendation page renamed to More Service

- Recommendation page was renamed to More Service.
- Fair Price Average controls were added.
- The page was split into service sections such as fair price average and recommendation.

14. Fair price average feature

- A monthly fair price snapshot design was introduced.
- SQL functions and queries were discussed to compute monthly averages by:
 - month
 - transaction type
 - property type
 - property city
 - bedrooms

- Matching app UI was added with selectable month.
- The app was wired to fetch the matching card from 'fair_price_averages'.
- Several SQL fixes were made for ambiguity and test data issues.

15. Seed/test property data

- Large SQL batches were provided to insert hundreds of property rows.
- Data was spread across multiple states and cities.
- Owner IDs were distributed across four provided users.
- Search/filter and RPC behavior with seeded data were debugged.

16. Deals and ratings

- Safer deal flow was recommended.
- Deal implementation required 'property_id' and database updates.
- Property status becomes inactive after a completed deal.
- Ratings after deal completion were discussed and implemented using 1-5 decimal values.
- RLS and schema issues around deals and ratings were debugged.

17. Notifications and wish matching

- SQL-based scheduled matching was originally created using 'cron.job' and 'match_wished_properties()' every 2 minutes.
- That scheduled SQL matcher was later identified and disabled to test AI matching instead.
- In-app only matching was preferred over push notifications or scheduled cloud automation.

18. AI matching attempt with OpenAI

- A Supabase Edge Function 'ai_match_recommendations' was created.
- The function authenticates the user, fetches recent wishes, fetches candidate properties, then uses an AI model to score matches and insert rows into 'notifications'.
- During testing, multiple issues were fixed:
 - improper code pasted as diff instead of raw source
 - missing secrets
 - authorization/JWT handling
 - CORS/OPTIONS handling
 - Responses API parameter updates
 - structured output schema updates
- Final blocker: OpenAI quota exhaustion.

19. Switch from OpenAI to Gemini

- The same edge-function idea was migrated to Gemini after OpenAI quota ran out.
- A Gemini API key was created and added as a secret.
- The function was rewritten to call 'gemini-2.5-flash:generateContent' with JSON schema output.
- Further testing was expected after the Gemini migration.

20. Add Property UX improvements

- All add-property fields were made mandatory.
- Draft saving was added so partially entered forms persist.

- Image preview and reorder were added.
- Input helpers were added for price and area units.
- Reordering was later adjusted so drag can start immediately without long press.

21. Admin dashboard setup

- A separate 'admin_web' project was scaffolded.
- Supabase '.env' setup was added with provided URL and anon key.
- Real counts from Supabase were wired into the dashboard.
- An admin login screen was added.
- Guidance was provided for creating an admin user and updating the role constraint so 'admin' is allowed.

22. Java/Gradle environment issues

- Java 8 install at 'C:\Program Files\Java\jre1.8.0_481' was found corrupted.
- A mismatch between Java 25 and Gradle caused 'Unsupported class file major version 69'.
- The working Java setup was moved to Temurin/OpenJDK 17.
- Guidance was provided to set 'JAVA_HOME', clean 'PATH', remove the broken JRE, and pin Gradle to Java 17.

23. Testing work

- Unit tests were added for auth and wished-property controller flows.
- 'flutter analyze' was brought to green for the targeted work.
- Testable hooks were added where needed, for example injecting a notification controller into wished-property flow.

24. Latest explanatory work

- A detailed explanation was provided for login and registration logic.
- A simple text flow diagram was provided for auth.
- A detailed explanation was also provided for owner add property, update, and delete flows.

Key files worth opening first

Mobile app auth

- 'AMAN/mobile_app/lib/features/auth/presentation/login_page.dart'
- 'AMAN/mobile_app/lib/features/auth/presentation/register_page.dart'
- 'AMAN/mobile_app/lib/features/auth/state/auth_controller.dart'
- 'AMAN/mobile_app/lib/features/auth/data/auth_repository.dart'

Mobile app properties

- 'AMAN/mobile_app/lib/add_property_page.dart'
- 'AMAN/mobile_app/lib/follow_up_property_page.dart'
- 'AMAN/mobile_app/lib/update_property_page.dart'
- 'AMAN/mobile_app/lib/features/properties/state/add_property_controller.dart'
- 'AMAN/mobile_app/lib/features/properties/state/update_property_controller.dart'
- 'AMAN/mobile_app/lib/features/properties/state/follow_up_properties_controller.dart'
- 'AMAN/mobile_app/lib/features/properties/data/property_repository.dart'

Mobile app wished/notifications/deals

- 'AMAN/mobile_app/lib/features/wished/...'
- 'AMAN/mobile_app/lib/features/notifications/...'
- 'AMAN/mobile_app/lib/features/deals/...'

Admin web

- 'AMAN/admin_web/src/App.jsx'
- 'AMAN/admin_web/src/lib/supabase.js'
- 'AMAN/admin_web/.env'

Edge Function

- 'AMAN/supabase/functions/ai_match_recommendations/index.ts'

Important operational notes

- Another Codex session will not automatically inherit this exact conversation memory from a PDF alone.
- The most effective handoff is: codebase + this document + any SQL schema/functions currently active in Supabase.
- If continuing AI matching work, verify the current deployed edge function, secrets, and whether Gemini is active.
- If continuing admin work, verify RLS and admin-role handling in both Auth and the 'public.user' table.

Suggested handoff instruction for the next user

A good first prompt for another Codex session is:

"Read 'CHAT_HANDOFF.md' first, then inspect the mobile auth flow, property repository, and admin dashboard setup. Continue from the current AMAN state without undoing existing architecture."